

Aula 1: Introdução a Aplicações Web, Sistemas Distribuídos e Maven

Bem-vindos à primeira aula! Este material de leitura foi preparado para dar a vocês a base teórica necessária para as nossas atividades práticas de hoje.

1. Aplicações Web e Sistemas Distribuídos

Uma **Aplicação Web** é um sistema de software que roda em um servidor e é acessado através de um navegador (browser) ou outro cliente através de uma rede (geralmente a Internet).

Um **Sistema Distribuído** é um conjunto de componentes independentes localizados em diferentes máquinas conectadas em rede, que se comunicam e coordenam suas ações através da troca de mensagens. Para o usuário, aparece como um sistema único e coerente. As aplicações web modernas, em sua essência, são sistemas distribuídos complexos.

2. Arquitetura e Modelos: Monolitos vs. Microsserviços

Existem diferentes abordagens para estruturar um sistema:

- **Arquitetura Monolítica:** Toda a aplicação (interface, lógica de negócio, acesso a dados) é construída, empacotada e implementada como uma única unidade indivisível. É mais simples de iniciar, mas torna-se difícil de escalar e manter à medida que o código cresce.
- **Microsserviços (Microservices):** A aplicação é dividida em um conjunto de serviços pequenos, independentes e focados. Cada serviço roda em seu próprio processo e se comunica através de mecanismos leves (geralmente APIs HTTP). Facilita a escalabilidade independente, o isolamento de falhas e a utilização de tecnologias adequadas para cada problema.

3. O Protocolo HTTP

O **HTTP (Hypertext Transfer Protocol)** é o protocolo base de comunicação da World Wide Web. Funciona em um modelo estrito de **Cliente-Servidor**: o cliente (ex: o seu navegador ou um aplicativo mobile) faz uma *Requisição* (Request), e o servidor processa e devolve uma *Resposta* (Response).

Verbos HTTP (Métodos)

Os verbos indicam a intenção da ação que queremos realizar no servidor. Em APIs RESTful, eles mapeiam diretamente as operações de CRUD (Create, Read, Update, Delete):

- **GET:** Solicita a representação de um recurso. Usado apenas para obter dados (Leitura).

- **POST:** Envia dados ao servidor para criar um novo recurso.
- **PUT:** Substitui todas as representações atuais do recurso de destino pelos dados da requisição (Atualização completa).
- **PATCH:** Aplica modificações parciais a um recurso (Atualização parcial).
- **DELETE:** Remove o recurso especificado.
- **OPTIONS:** Retorna os métodos HTTP suportados pelo servidor para uma URL específica (muito usado no CORS).

Status HTTP (Códigos de Resposta)

Sempre que o servidor responde, ele envia um código de status para indicar o resultado da requisição:

- **1xx (Informativos):** A requisição foi recebida e o processo continua.
- **2xx (Sucesso):** A ação foi recebida com sucesso, compreendida e aceita.
 - *Exemplos:* 200 OK , 201 Created , 204 No Content .
- **3xx (Redirecionamento):** Uma ação adicional precisa ser tomada pelo cliente para concluir a requisição.
- **4xx (Erro do Cliente):** A sintaxe da requisição está incorreta ou não pode ser processada.
 - *Exemplos:* 400 Bad Request , 401 Unauthorized (falta de autenticação), 403 Forbidden (falta de permissão), 404 Not Found .
- **5xx (Erro do Servidor):** O servidor falhou ao cumprir uma requisição válida devido a um problema interno.
 - *Exemplos:* 500 Internal Server Error , 503 Service Unavailable .

4. Maven: Gerenciamento de Dependências

O **Apache Maven** é uma ferramenta de gerenciamento de configuração e automação de *build* focada em projetos Java. Ele é responsável por baixar e gerenciar dependências (bibliotecas de terceiros), compilar o código, executar testes e empacotar o software de forma padronizada.

O coração de um projeto Maven é o arquivo `pom.xml` (Project Object Model), onde declaramos o que o nosso projeto precisa para funcionar.

Comandos Mais Utilizados

-  `mvn clean` : Limpa o diretório de compilação (remove a pasta `target/`), garantindo um *build* limpo.
-  `mvn compile` : Compila o código-fonte principal do projeto.
-  `mvn test` : Executa os testes unitários do projeto.
-  `mvn package` : Compila, testa e empacota o código em um formato distribuível, como um arquivo `.jar` .
-  `mvn install` : Instala o pacote criado no seu repositório local, tornando-o disponível como dependência para outros projetos locais.

5. Prática com a Dependência Jackson (Preparação)

Durante a nossa parte prática de hoje, vamos ver como o Maven facilita a nossa vida ao adicionar bibliotecas sem precisarmos gerenciar arquivos `.jar` manualmente.

Vamos utilizar o **Jackson**, que é a principal biblioteca em Java para processar dados no formato **JSON** (JavaScript Object Notation). O Jackson é responsável por:

1. **Serialização**: Converter objetos Java em uma string JSON.
2. **Desserialização**: Converter uma string JSON de volta para objetos Java.

Na parte prática, iremos adicionar o Jackson ao nosso `pom.xml` e escrever algumas linhas de código para ver essa transformação acontecer, nos preparando para criar os nossos primeiros *endpoints* em Quarkus nas próximas aulas!